

Menganalisis 2 style: Struktural dan OOP pada Soal Eolymp #26

link soal: eolymp.com/en/problems/26

Identifikasi Masalah

Ada dua orang:

- Snow-maiden: tugasnya membungkus hadiah (packing).
- Santa Claus: tugasnya mengantarkan hadiah (delivery).

Ada n hadiah yang harus diberikan kepada anak-anak.

Untuk setiap hadiah ke- i :

- Waktu yang dibutuhkan Snow-maiden untuk membungkus: $t1[i]$
- Waktu yang dibutuhkan Santa untuk mengantar: $t2[i]$

Setiap hadiah harus melalui dua tahap berurutan:

- Tahap 1: Snow-maiden membungkus (tidak bisa langsung diantar sebelum dibungkus).
- Tahap 2: Santa mengantar (setelah hadiah selesai dibungkus).

Aturannya:

- Snow-maiden hanya bisa membungkus satu hadiah dalam satu waktu.
- Santa hanya bisa mengantar satu hadiah dalam satu waktu.
- Santa tidak bisa mulai mengantar hadiah ke- i sebelum dibungkus selesai.

Penyelesaian

1. Struktural

```
#include <stdio.h>
#include <vector>
#include <algorithm>
using namespace std;

typedef struct Struct_Jobs{
    int t1, t2;
} jobs;

bool comp(jobs a, jobs b){
    return min(a.t1, b.t2) < min(a.t2, b.t1);
}

int main(){
    int i, n, temp1[305], temp2[305], packing, res;
    vector<jobs> v_jobs;

    scanf("%d", &n);
    for(i = 0; i < n; i++) scanf("%d", &temp1[i]);
```

```

for(i = 0; i < n; i++) scanf("%d", &temp2[i]);
for(i = 0; i < n; i++) {
    jobs temp;
    temp.t1 = temp1[i];
    temp.t2 = temp2[i];
    v_jobs.push_back(temp);
}
packing = 0; res = 0;
sort(v_jobs.begin(), v_jobs.end(), comp);
for(i = 0; i < n; i++){
    packing += v_jobs[i].t1;
    if(packing > res) res = packing;
    res += v_jobs[i].t2;
}

printf("%d\n", res);

return 0;
}

```

2. Object Oriented Programming (OOP)

```

#include <stdio.h>
#include <vector>
#include <algorithm>
using namespace std;

template<typename T>
T getNum(){
    T res = 0, b = 0;
    char c;

    while(1){
        c=getchar_unlocked();
        if(c == '-') b = 1;
        if(c == ' ' || c == '\n') continue;
        else break;
    }
    if(c!='-') res = c - '0';
    while(1){
        c=getchar_unlocked();
        if(c >= '0' && c <= '9') res = 10 * res + c - '0';
        else break;
    }
    if(b == 1) res *= -1;
    return res;
}

```

```

}

class Jobs {
private:
    int t1, t2;

public:
    Jobs (int a, int b): t1(a), t2(b) {}
    int getPacking() { return t1; }
    int getDelivery() { return t2; }
    bool operator < (const Jobs &b) const {
        return min(t1, b.t2) < min(t2, b.t1);
    }
};

int main(){
    int i, n, temp1[305], temp2[305], packing, res;
    vector<Jobs> v_jobs;

    n = getNum<int>();
    for(i = 0; i < n; i++) temp1[i] = getNum<int>();
    for(i = 0; i < n; i++) temp2[i] = getNum<int>();
    for(i = 0; i < n; i++) {
        Jobs temp (temp1[i], temp2[i]);
        v_jobs.push_back(temp);
    }
    packing = 0; res = 0;
    sort(v_jobs.begin(), v_jobs.end());
    for(i = 0; i < n; i++){
        packing += v_jobs[i].getPacking();
        if(packing > res) res = packing;
        res += v_jobs[i].getDelivery();
    }

    printf("%d\n", res);

    return 0;
}

```

Penjelasan Kode

1. Perbedaan antara kode 1 dan kode 2:

Gaya Pemrograman:

- Kode 1 menggunakan gaya prosedural.

- Kode 2 menggunakan gaya berorientasi objek (OOP) dengan class dan enkapsulasi.

Struktur Data:

- Kode 1 memakai struct yang hanya menyimpan data (t1, t2).
- Kode 2 memakai class Jobs dengan atribut private dan fungsi getter, sehingga lebih aman dan modular.

Fungsi Pembandingan untuk Sort:

- Kode 1 menggunakan fungsi global comp() yang dikirim ke sort.
- Kode 2 menggunakan operator overloading < di dalam class, jadi sort() bisa langsung dipanggil tanpa argumen tambahan.

Input Data:

- Kode 1 memakai scanf, cara standar dan mudah dibaca.
- Kode 2 memakai fungsi getNum() dengan getchar_unlocked(), yaitu metode input cepat yang sering dipakai di kompetisi programming.

Akses Data:

- Kode 1 langsung akses v_jobs[i].t1 dan v_jobs[i].t2.
- Kode 2 akses lewat fungsi getPacking() dan getDelivery(), sesuai prinsip OOP.

Efisiensi dan Keterbacaan:

- Kode 1 lebih sederhana dan cocok untuk belajar dasar logika program.
- Kode 2 lebih cepat dalam input besar, dan lebih mudah dikembangkan dalam sistem besar.

Kesimpulan:

- Kode 1 versi sederhana dan mudah dibaca.
- Kode 2 versi lebih cepat, modern, dan rapi secara struktur OOP.

2. Alur Logika Kode

Secara garis besar alur logika dari kode 1 dan kode 2 sama, hanya berbeda dari implementasi pemrogramannya saja. berikut alur dari logika kode penyelesaian soal:

Input jumlah pekerjaan (n): Program membaca banyaknya pekerjaan yang akan diproses.

Input dua deret waktu:

- t1[i]: waktu packing (proses awal) untuk pekerjaan ke-i
- t2[i]: waktu delivery (proses kedua) untuk pekerjaan ke-i

Jadi setiap pekerjaan punya dua tahap: packing -> delivery.

Menyimpan data pekerjaan: Setiap pekerjaan disimpan dalam bentuk objek/struct yang berisi t1 dan t2, lalu dimasukkan ke dalam vector.

Mengurutkan pekerjaan (sorting):

Pekerjaan disortir dengan aturan:

$\min(t1, b.t2) < \min(t2, b.t1);$

penjelasan:

Setiap pekerjaan memiliki dua tahap, yaitu tahap packing ($t1$) dan tahap delivery ($t2$). Karena kedua tahap ini harus dilakukan secara berurutan dan tidak bisa bersamaan untuk pekerjaan yang sama, maka urutan pengerjaan pekerjaan sangat berpengaruh terhadap waktu total penyelesaian. Untuk menentukan urutan terbaik, program menggunakan fungsi pembandingan dengan rumus diatas. Rumus ini membandingkan dua pekerjaan, A dan B, untuk memutuskan mana yang lebih efisien dikerjakan lebih dahulu. Jika nilai minimum antara waktu packing A dan waktu delivery B lebih kecil daripada nilai minimum antara waktu delivery A dan waktu packing B, maka pekerjaan A akan ditempatkan sebelum pekerjaan B. Intuisi di balik perbandingan ini adalah untuk mengurangi waktu menganggur (idle time) antara tahap packing dan delivery, sehingga kedua tahap dapat bekerja seimbang tanpa saling menunggu terlalu lama. Dengan cara ini, urutan hasil sorting menghasilkan alur kerja yang paling efisien dan meminimalkan total waktu penyelesaian seluruh pekerjaan.

Menghitung total waktu penyelesaian:

- Inisialisasi packing = 0 dan res = 0
- Untuk setiap pekerjaan hasil urutan:
 - Tambahkan $t1$ ke packing: artinya proses packing pekerjaan tersebut dimulai setelah semua packing sebelumnya selesai.
 - Jika packing lebih besar dari res, berarti delivery harus menunggu packing: res disesuaikan ke nilai packing.
 - Tambahkan $t2$ ke res: menandakan waktu delivery pekerjaan ini selesai.

Output hasil akhir: Setelah semua pekerjaan diproses, nilai res adalah total waktu penyelesaian seluruh pekerjaan.